

Build a Production-Ready Minimal Social Publishing Application on AWS Amplify

Act as a senior full-stack engineer and AWS solutions architect.

Build a **production-ready, minimal social publishing application** where an administrator can publish articles/opinions and public users can read them.

You may choose a short, professional, memorable product name that fits a platform for publishing opinions and articles.

1. Core Principle

The entire application must be managed from this repository.

I do **not** want to manually create, modify, or configure AWS resources through the AWS Console.

Any infrastructure required by the application must be:

- Defined in code.
- Version-controlled in this repository.
- Provisioned and updated through AWS Amplify or the infrastructure mechanism supported by the existing Amplify project.
- Reusable across deployments.
- Idempotent: redeploying the same repository must update/reuse existing infrastructure instead of creating duplicate resources.

Do not require manual AWS infrastructure configuration unless there is absolutely no programmatic alternative.

If some unavoidable one-time AWS action is required, clearly document it rather than silently assuming it has already been completed.

2. AWS Region

The application is primarily for users in India.

Use an AWS region close to Indian users.

Prefer:

`ap-south-1` — Mumbai

Keep the region configurable where appropriate, but use Mumbai as the default.

3. Application Scope

Keep the first version deliberately small.

There are only two major application capabilities:

A. Public Feed

Anyone visiting the application can:

- View published articles.
- View the newest articles first.
- Open/read an article.
- Use the application without creating an account.
- Use the application without logging in.

Public users are **read-only users**.

They must not be able to:

- Create articles.
- Edit articles.
- Delete articles.
- Access admin functionality.

B. Admin Article Management

There must be a protected admin area.

Only an authenticated administrator can:

- Log in.
- Create an article.
- Publish an article.
- View articles from the admin interface.

For the initial version, focus primarily on **creating and publishing articles**.

Do not unnecessarily add:

- Likes
- Comments
- Followers
- User profiles
- Messaging
- Notifications
- Social connections
- Complex moderation systems

The product should stay minimal.

4. Authentication

Use **AWS Cognito** for authentication.

Important:

- Authentication is required **only for administrators**.
- Public readers must not need Cognito accounts.
- Protect all admin routes.
- Protect backend article-creation operations.
- Do not rely only on frontend route protection.
- The backend must independently verify authorization before allowing article creation.

Design the authorization model so an authenticated normal Cognito user cannot automatically become an administrator.

Prefer an appropriate Cognito group/role/claim-based authorization mechanism for determining admin access.

For example:

ADMIN

Do not hard-code credentials, passwords, user IDs, access keys, or secrets inside the source code.

5. Article Storage

Use **Amazon DynamoDB** to store published articles unless the existing Amplify architecture provides an equivalent AWS-native managed persistence abstraction that clearly maps to DynamoDB.

Create a stable logical article resource.

Conceptually, the table represents:

articles

The deployment must be idempotent.

For example:

If the first deployment creates the article storage resource, the second deployment must **reuse/update that same resource**.

It must NOT create resources such as:

- articles-1

- articles-2
- articles-new
- articles-random-id

simply because another deployment occurred.

Use stable infrastructure definitions and logical identifiers.

Do not dynamically generate infrastructure resource names on every deployment.

6. Suggested Article Data Model

Keep the model simple.

An article should contain approximately:

```
id
title
slug
summary
content
createdAt
updatedAt
publishedAt
status
author
```

Recommended status values:

```
DRAFT
PUBLISHED
```

For the public feed, return only published articles.

Sort articles with the newest published article first.

Use timestamps in a consistent standard format such as UTC ISO-8601.

Generate IDs safely using UUID/ULID or an equivalent collision-resistant mechanism.

If a different DynamoDB key/index design would provide better query performance, implement it and briefly document why.

Avoid DynamoDB table scans for the primary feed if a better access pattern can reasonably be implemented.

7. Minimum UI

Build only the interfaces required for the MVP.

Public Feed

Create a polished homepage/feed showing published articles.

Each feed item should preferably show:

- Article title
- Short description/summary
- Publication date
- Author
- Article content or a way to open the full article

The UI must be:

- Minimalistic
- Modern
- Responsive
- Fast
- Accessible
- Mobile-friendly
- Smooth
- Professional

Use reusable UI components.

For example:

```
Layout
Header
ArticleCard
ArticleList
LoadingState
EmptyState
ErrorState
Button
Input
Textarea
```

Do not duplicate UI logic unnecessarily.

8. Admin UI

Create an admin login page.

After successful authentication, provide a simple article creation interface.

The create-article form should include at least:

```
Title
Summary
Content
Publish action
```

Provide appropriate:

- Validation
- Loading states
- Success states
- Error states
- Disabled states while submitting

Prevent duplicate submissions.

After an article is successfully published, it should become visible on the public feed.

9. Routing

A simple route structure could be:

```
/
  Public article feed

/article/:slug
  Public article page

/admin/login
  Admin login

/admin
  Protected admin dashboard

/admin/articles/new
  Create article
```

You may adjust the exact routes if the selected framework has a cleaner convention.

10. Technology Choice

Inspect the existing repository before deciding the implementation.

Do not replace the project's framework unnecessarily.

If the repository is empty or there is no strong existing architecture, choose a modern, stable stack suitable for AWS Amplify.

Prefer:

- TypeScript
- React / Next.js or the framework best supported by the existing repository
- AWS Amplify
- AWS Cognito
- DynamoDB
- AWS-managed backend services

Avoid adding unnecessary dependencies.

Do not introduce another cloud provider.

11. Repository-First Infrastructure

This requirement is extremely important.

Everything possible must live in this repository, including:

- Authentication configuration
- Database configuration
- Backend configuration
- Authorization
- Environment configuration
- Infrastructure definitions
- Deployment-related configuration

A fresh developer should be able to:

1. Clone the repository.
2. Install dependencies.
3. Configure only the required deployment/account credentials or documented environment variables.
4. Deploy the application.
5. Have the required infrastructure provisioned automatically.

Do not design a solution where the developer must manually create Cognito pools, DynamoDB tables, APIs, IAM policies, etc. through the AWS Console.

12. Deployment Idempotency

Infrastructure deployments must be repeatable.

Running deployment multiple times must result in:

```
Deployment 1:  
Create required resources.  
  
Deployment 2:  
Reuse/update existing resources.  
  
Deployment 3:  
Reuse/update existing resources.
```

It must NOT behave like:

```
Deployment 1:  
Create articles table.  
  
Deployment 2:  
Create another articles table.  
  
Deployment 3:  
Create another articles table.
```

Follow Infrastructure-as-Code principles.

Use stable resource definitions.

13. Security Requirements

Treat this as production code.

Implement:

- Least-privilege IAM permissions.
- Backend authorization for write operations.
- Cognito-based admin authentication.
- Protected admin routes.
- Input validation.
- Output encoding where appropriate.
- Secure handling of environment variables.
- No AWS secrets committed to Git.
- No credentials in frontend bundles.

- No sensitive information in logs.
- Safe error handling.
- HTTPS-compatible architecture.
- Reasonable security headers where supported.

Public users should receive only the permissions required to read published articles.

Administrative write operations must require validated administrator authorization.

14. Reliability

Implement sensible production safeguards without overengineering.

Where applicable:

- Handle API errors gracefully.
 - Handle expired authentication sessions.
 - Prevent duplicate article creation caused by repeated clicks.
 - Use deterministic/idempotent write behavior where appropriate.
 - Display meaningful frontend error states.
 - Add reasonable request timeouts if custom network requests are introduced.
 - Avoid infinite retry loops.
-

15. Performance

The application should load quickly.

Implement sensible optimizations such as:

- Efficient DynamoDB access patterns.
- Pagination when appropriate.
- Avoiding unnecessary network requests.
- Avoiding unnecessary React re-renders.
- Code splitting where supported automatically by the framework.
- Appropriate caching for public content where safe.
- Optimized production builds.

Do not introduce Redis, OpenSearch, Kafka, or other infrastructure for this MVP.

Keep the architecture simple.

16. Code Quality

Use a modular and maintainable project structure.

Follow:

- DRY
- SOLID where appropriate
- Separation of concerns
- Type safety
- Clear naming
- Small reusable components
- Centralized configuration
- Consistent error handling

Prefer shared abstractions over copying the same logic into multiple files.

For example, do not implement Cognito access independently in multiple components if a reusable authentication service/hook/module can handle it centrally.

17. Testing

Add meaningful tests for critical functionality.

At minimum test:

- Article validation.
- Article creation logic.
- Unauthorized users cannot create articles.
- Admin authorization.
- Published article retrieval.
- Public feed behavior.
- Important reusable UI/business logic where practical.

Do not write meaningless tests solely to increase coverage.

18. UX States

Every important screen must handle:

Loading

Success

Empty

Validation error

Network/backend error

Unauthorized

The application should never leave the user looking at a blank screen because an API request failed.

19. Logging

Use structured logging on the backend where appropriate.

Log enough information to diagnose failures, but never log:

- Passwords
 - Authentication tokens
 - AWS credentials
 - Secrets
 - Sensitive Cognito information
-

20. Cost

Keep AWS costs very low for the initial version.

Prefer:

- Serverless services.
- Pay-per-use infrastructure.
- Minimal always-running infrastructure.
- DynamoDB on-demand capacity where appropriate.
- Amplify-managed/serverless deployment.

Avoid provisioning infrastructure that is unnecessary for the MVP.

21. UI Design Direction

The website should feel like a premium editorial/opinion publishing platform rather than a generic CRUD dashboard.

Design characteristics:

- Clean typography.
- Generous whitespace.
- Strong article titles.
- Excellent readability.
- Subtle animation/transitions.
- Clear hierarchy.
- Minimal visual clutter.
- Professional desktop experience.
- Excellent mobile experience.

Do not overuse gradients, animations, cards, shadows, or decorative elements.

Content should remain the primary focus.

22. Before Implementing

First inspect the repository completely.

Understand:

- Existing framework.
- Existing Amplify configuration.
- Existing package dependencies.
- Existing authentication configuration.
- Existing infrastructure.
- Existing deployment setup.
- Existing coding conventions.

Reuse what is already present whenever reasonable.

Do not blindly initialize another application inside the repository.

Do not delete existing working infrastructure unless replacing it is necessary and justified.

23. Implementation Approach

After inspecting the repository:

1. Briefly explain the architecture you intend to implement.
2. Identify which existing files/resources will be reused.
3. Identify which AWS resources are required.
4. Implement the application.
5. Configure infrastructure through the repository.
6. Add authentication and authorization.
7. Add DynamoDB-backed article persistence.
8. Build the public feed.
9. Build the admin article creation interface.
10. Add validation, error handling, and security controls.
11. Add tests.
12. Run available linting, type-checking, tests, and production builds.
13. Fix issues discovered by those checks.
14. Provide final deployment/documentation instructions.

Do not stop after generating an architecture proposal. Implement the actual working code.

24. Important Constraints

DO NOT:

- Ask me to manually create a DynamoDB table.
- Ask me to manually create a Cognito User Pool.
- Ask me to manually create IAM roles unless absolutely unavoidable.
- Store AWS access keys in source code.
- Hard-code administrator passwords.
- Make public users log in.
- Add unnecessary social-media features.
- Introduce unnecessary microservices.
- Create infrastructure with random names on every deployment.
- Duplicate resources during redeployment.
- Add unnecessary AWS services.
- Overengineer the MVP.
- Leave placeholder implementations for core functionality.

If AWS Amplify requires a particular infrastructure pattern, follow the Amplify-recommended approach while maintaining these constraints.

25. Definition of Done

The implementation is complete when I can deploy the repository and achieve the following flow:

1. Open website
- ↓
2. See public article feed without authentication
- ↓
3. Navigate to admin login
- ↓
4. Authenticate using Cognito
- ↓
5. Cognito authorization confirms administrator access
- ↓
6. Create an article
- ↓
7. Article is persisted in DynamoDB
- ↓
8. Article becomes visible in the public feed
- ↓
9. Deploy the same repository again
- ↓
10. Existing Cognito/DynamoDB resources are reused or updated, rather than duplicated

The resulting repository must be suitable as a **production-quality MVP**, not merely a proof of concept.

26. Final Documentation

At the end, provide a concise README section covering:

Architecture

Explain the frontend-to-backend flow and AWS services being used.

AWS Resources

List every AWS resource created by this repository and why it exists.

Local Development

Provide exact commands required to run the project locally.

Deployment

Provide exact commands required to deploy it through Amplify.

Admin Setup

Explain the correct and secure process for creating/designating the initial administrator.

Do not put an administrator password in source code.

Environment Variables

Document every required variable and distinguish between:

- Public frontend variables.
- Server-side variables.
- Secrets.

Verification

Explain how to verify:

- Public feed access.
 - Admin authentication.
 - Admin authorization.
 - Article creation.
 - DynamoDB persistence.
 - Redeployment/idempotency.
-

Priority Order

When making implementation decisions, optimize in this order:

1. **Security**
2. **Correctness**
3. **Infrastructure idempotency**
4. **Maintainability**
5. **User experience**
6. **Performance**
7. **Cost efficiency**
8. **Simplicity**

If you encounter an architectural ambiguity, select the simplest production-ready AWS-native solution that satisfies these requirements and document the decision rather than adding unnecessary complexity.